RICE SOCIAL SCIENCES
Department of Linguistics

neural networks

LING 430 Computational Linguistics Fall 2026
lecture 12a
Nov 10 2026

Siyu Liang
siyu.liang@rice.edu

logistics

- **HW 6 due Thursday, Nov 12, 11:59 pm**

last time

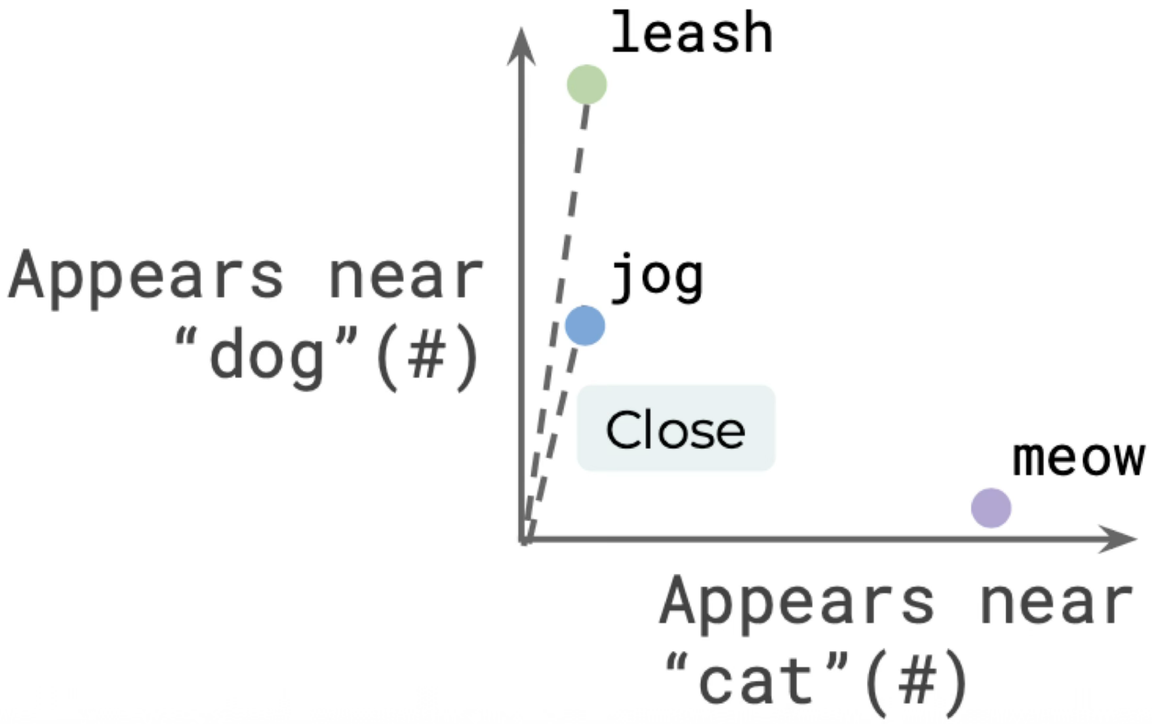
- word embeddings
  - sparse vs dense vectors
  - word2vec
  - properties of embeddings

agenda

- neural networks
  - neural units
  - the XOR problem
  - feedforward networks
- neural network applications
  - classification
  - neural language models

recap

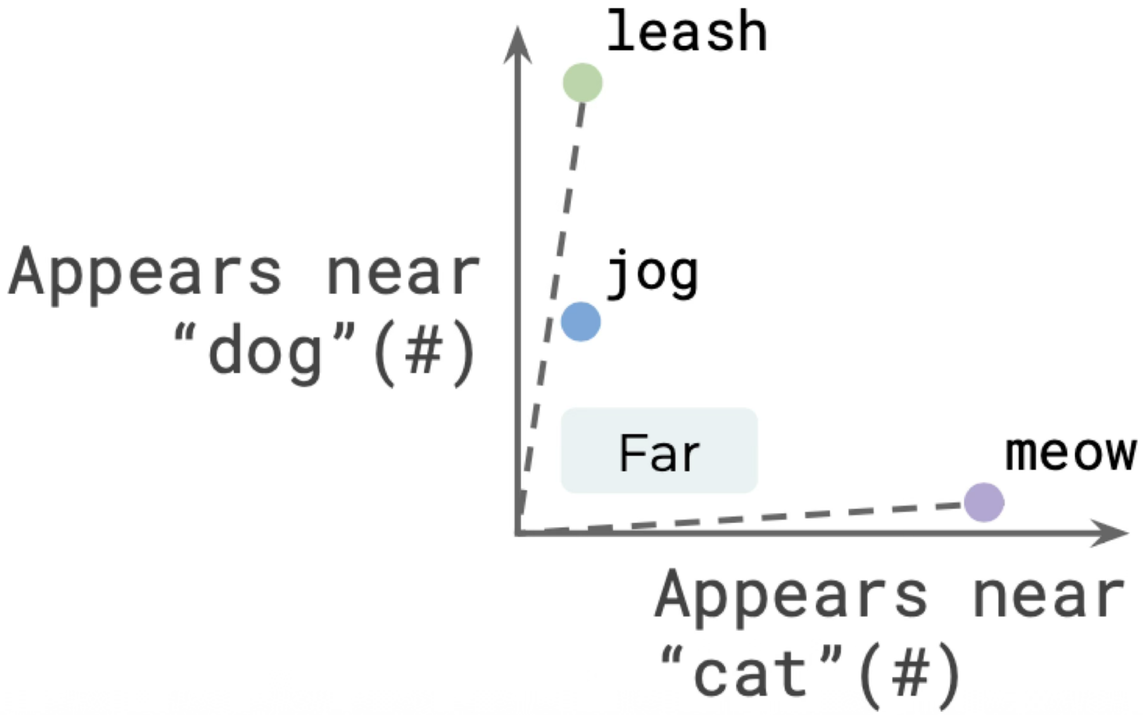
Example: another 2-D word vector space



A relevant distance metric here is **angle** – the dog:cat ratio

recap

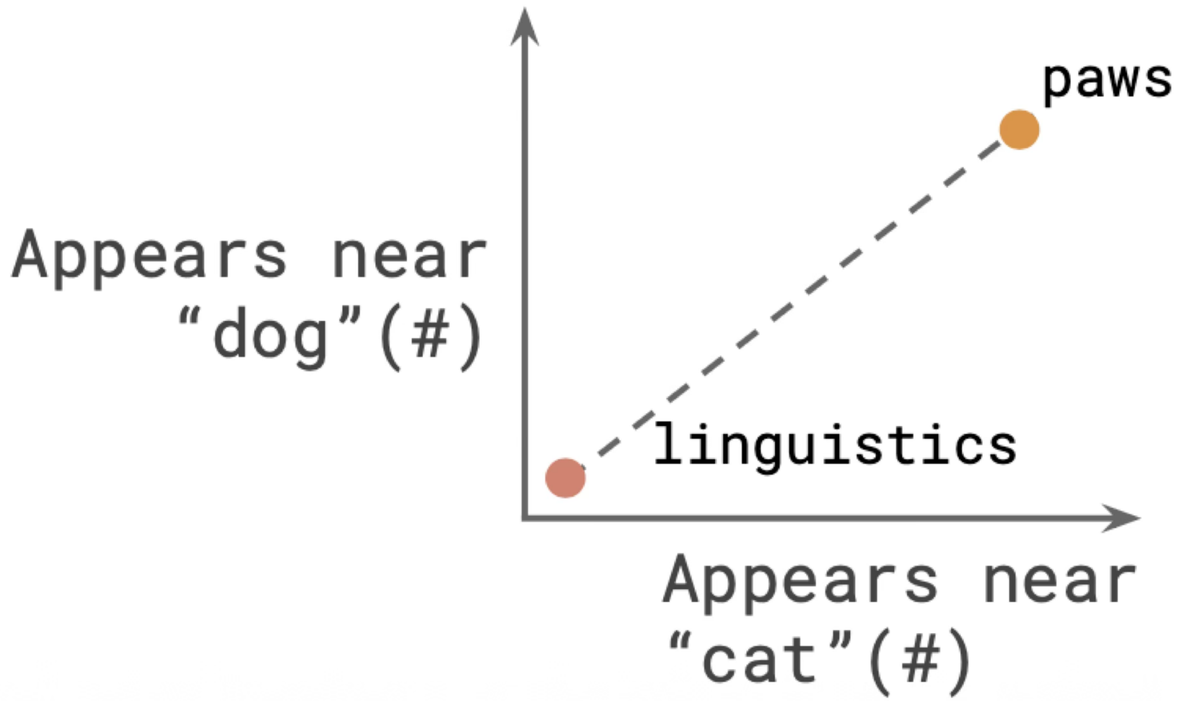
Example: another 2-D word vector space



A relevant distance metric here is **angle** – the dog:cat ratio

recap

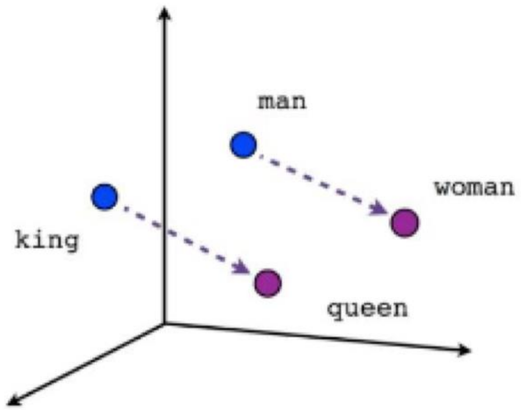
Example: another 2-D word vector space



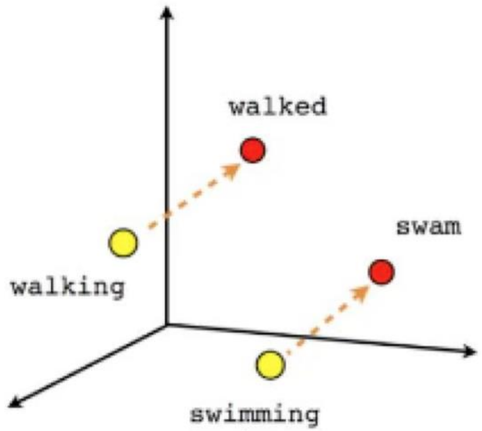
Distance is still a relevant metric!

recap

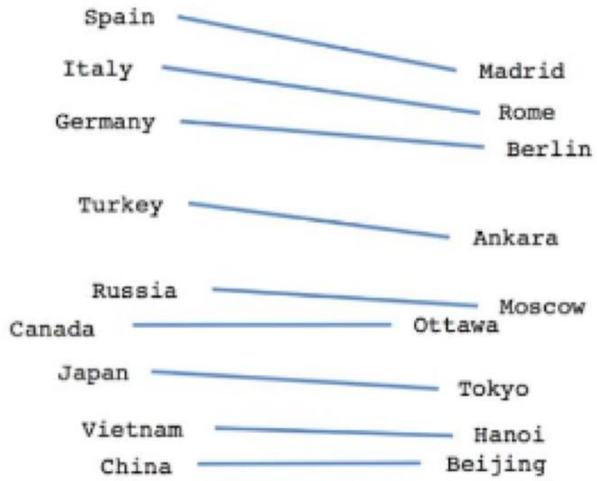
- projecting vectors to lower dimensions
  - reveal systematic relationships
  - word level similarity



Male-Female



Verb tense



Country-Capital

recap

- what's good about them?
- **word embeddings** are **vectors**, which can be inputs to computational models
- vector representations give models some sense of the nebulous idea of **semantic relatedness**

neural units

neural networks

recap

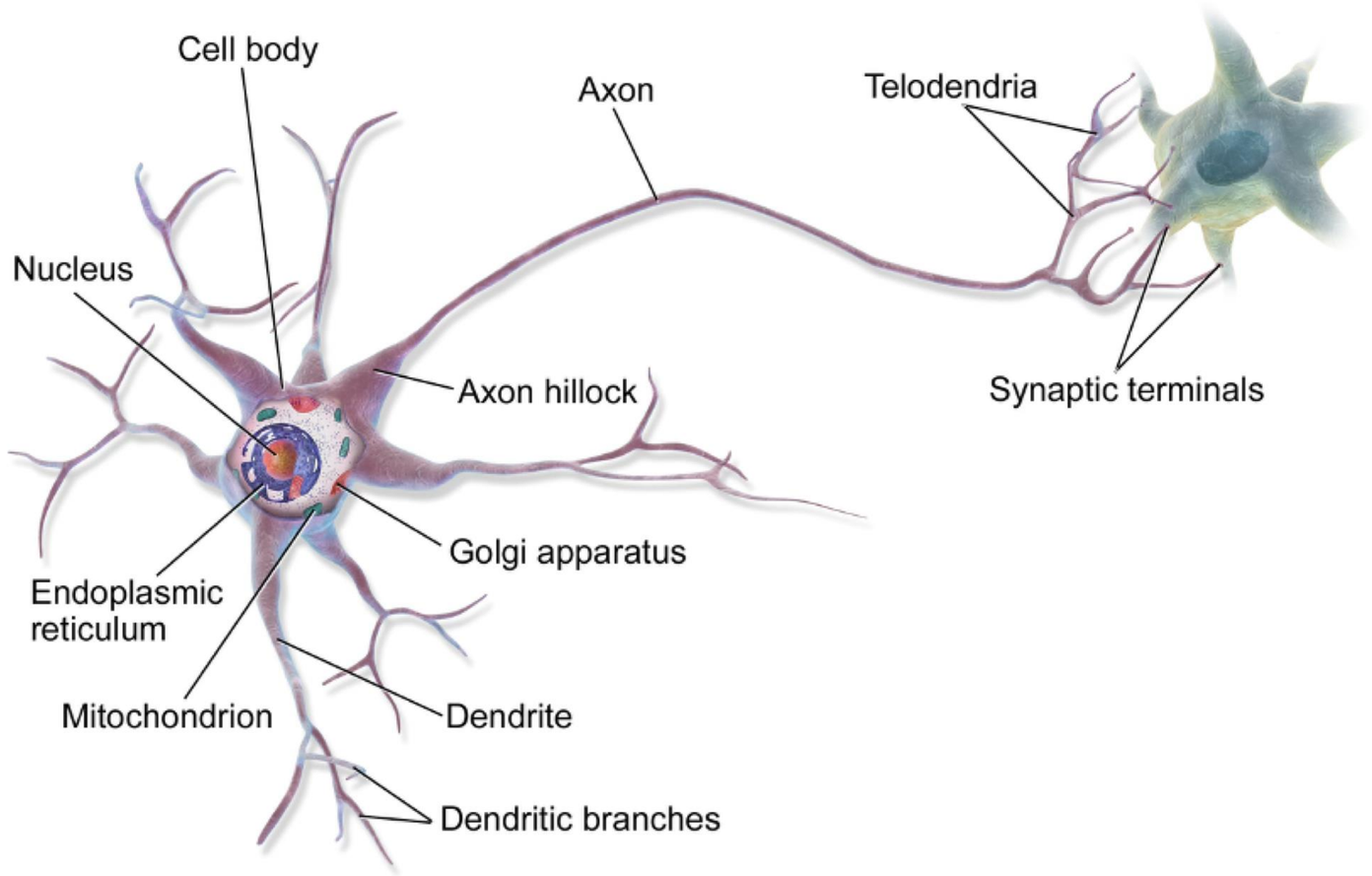
- two tasks we worked on in the class so far
  - classification (week 8): is this review positive or negative?
    - we used logistic regression, with features we designed by hand
  - language modeling (week 7): what is the next word?
    - we counted n-grams
- last week we learned to represent words as vectors
- can a model take those vectors as input, and learn the features on its own?

neural networks

- yes! that is what a **neural network** does
  - today we introduce a basic version for the same tasks
  - on Thursday we scale it up to the transformer, the network behind every large language model
- a neural network is made of small computing **units**
  - what are they?

this is in your brain

a neuron



by BruceBlaus - Own work, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=28761830

this is not in your brain

a “neuron”

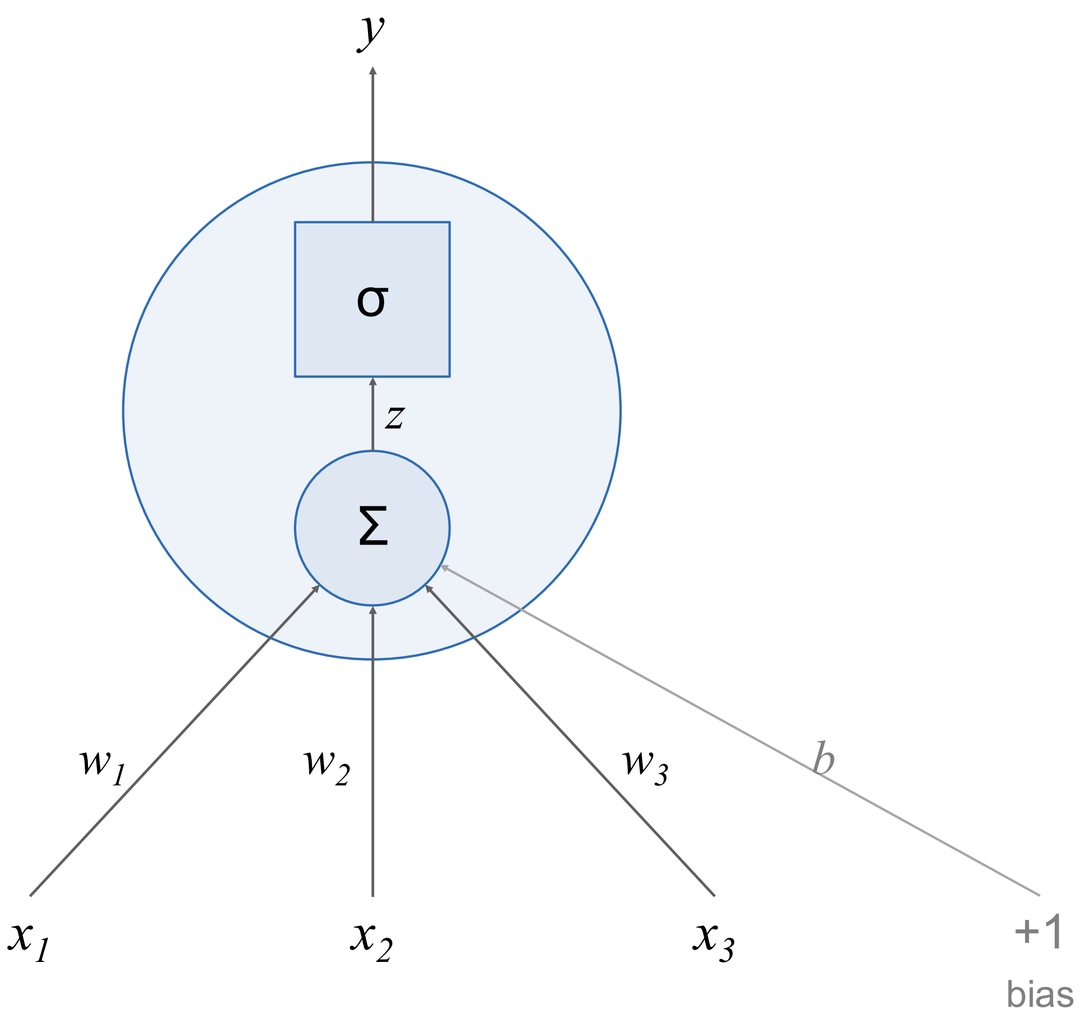
output value

non-linear activation function

weighted sum

weights

input layer



neurons

- neurons transmit excitation/inhibition to dendrites of other neurons
- activity above a threshold sends signals through axon (integrate & fire, Lapicque 1907(!))
- approximately **86 billion** neurons, **100 trillion** synapses in average human brain
- what about neural networks?
  - GPT-2 (2019): 1.5B parameters; GPT-3 (2020): 175B; Llama 3.1 (2024): 405B; DeepSeek V4 (2026): 1.6T; Kimi K3 (2026): **2.8T**
  - closed models like GPT-5, Claude and Gemini don’t publish their sizes

neural units

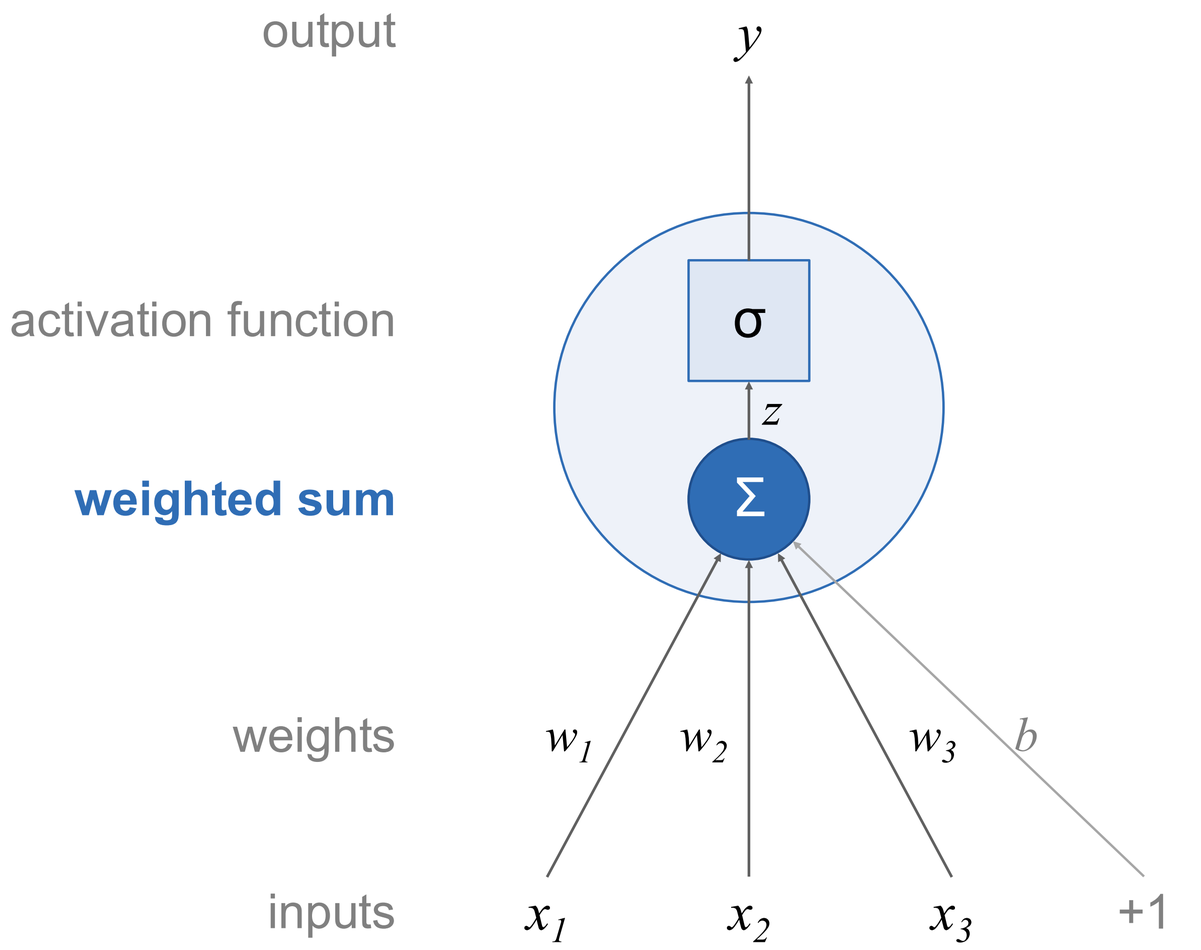
the weighted sum

- a unit has one **weight** for each input, plus a **bias**

$$z = b + \sum_i w_i x_i$$

- or

$$z = w \cdot x + b$$



neural units

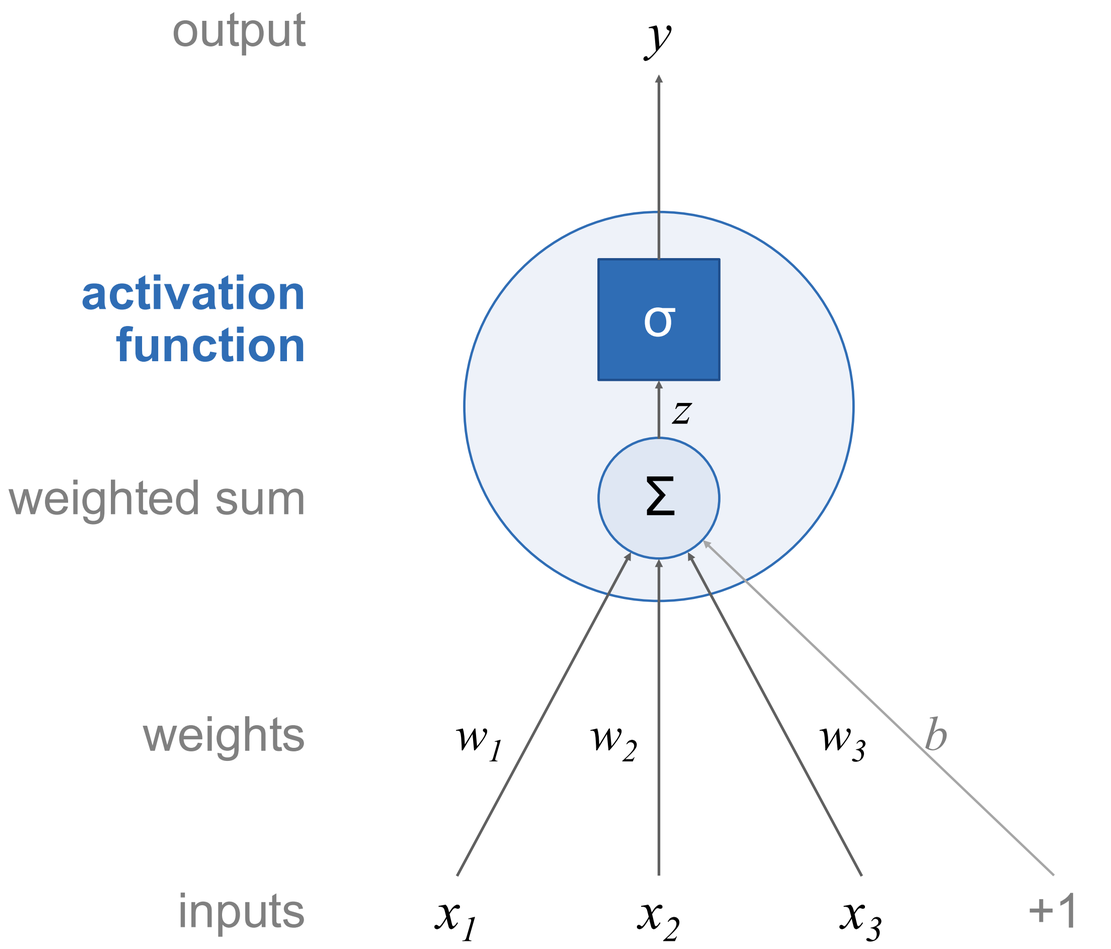
the activation

- the unit doesn't output z directly
- it first passes z through an **activation function** f

$$y = a = f(z)$$

- we already know one: the sigmoid

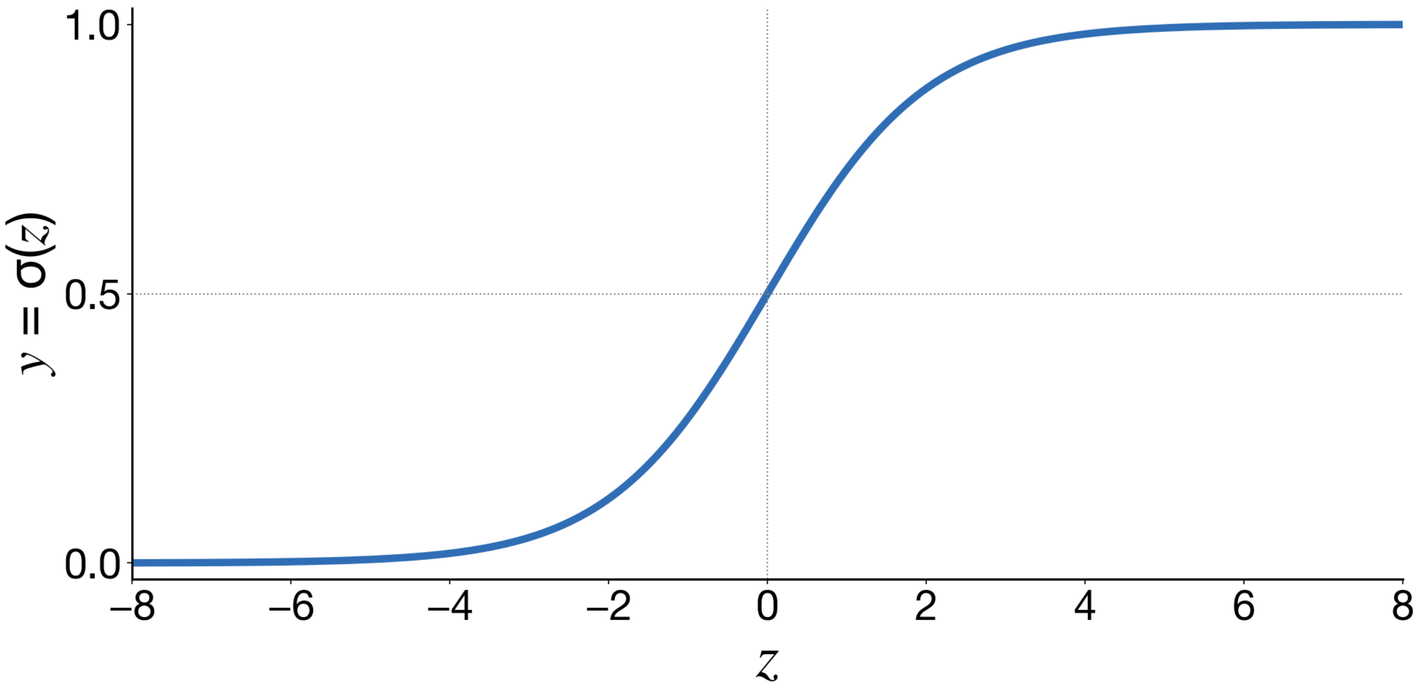
$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$



neural units

the sigmoid

- the sigmoid turns any number into a number between 0 and 1



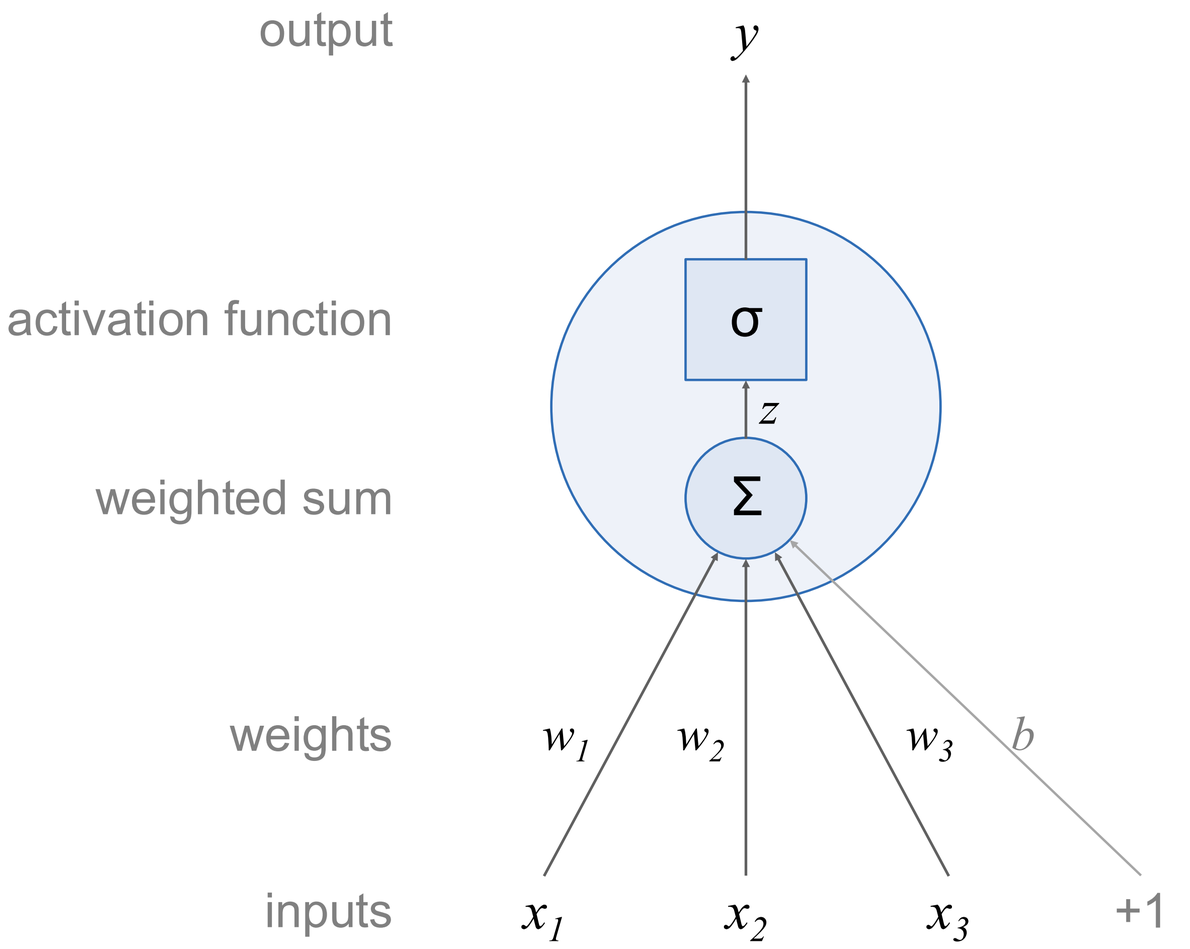
neural units

the whole unit

- putting the pieces together, a unit computes

$$y = \sigma(w \cdot x + b)$$

- a weighted sum
- then a bias
- then an activation function



neural units

an example

- suppose a unit has these weights and this bias
  $w = [0.2,0.3,0.9], b = 0.5$
- what does it output for this input?
  $x = [0.5,0.6,0.1]$

neural units

1. the weighted sum

$$w = [0.2, 0.3, 0.9], b = 0.5, x = [0.5, 0.6, 0.1]$$

- first, multiply each input by its weight and add them up
  $w \cdot x = 0.5 \times 0.2 + 0.6 \times 0.3 + 0.1 \times 0.9 = 0.10 + 0.18 + 0.09 = 0.37$

neural units

2. the bias

$$w = [0.2, 0.3, 0.9], b = 0.5, x = [0.5, 0.6, 0.1]$$

- first, multiply each input by its weight and add them up
  $w \cdot x = 0.5 \times 0.2 + 0.6 \times 0.3 + 0.1 \times 0.9 = 0.10 + 0.18 + 0.09 = 0.37$
- then add the bias
  $z = w \cdot x + b = 0.37 + 0.5 = 0.87$

neural units

3. the activation

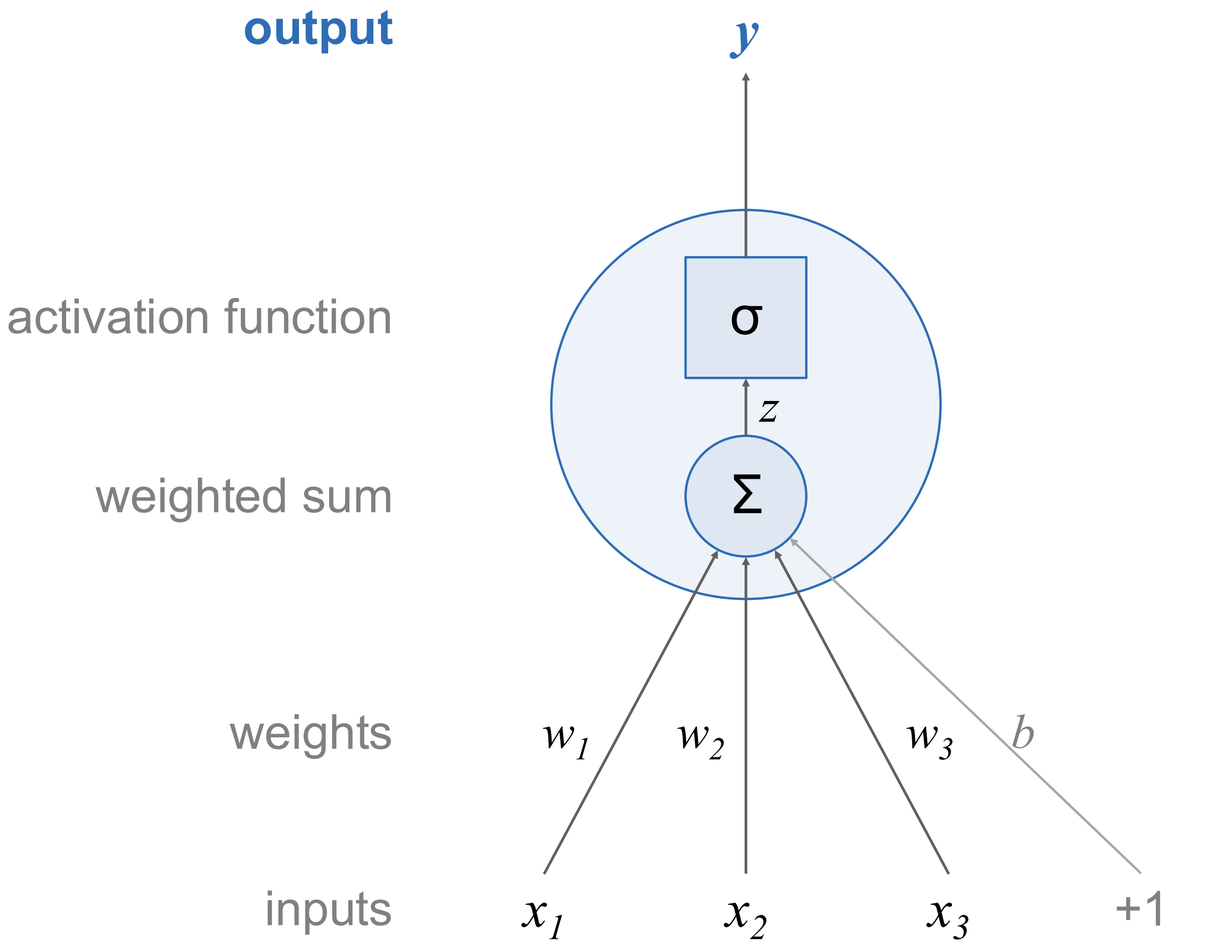
$$w = [0.2,0.3,0.9], b = 0.5, x = [0.5,0.6,0.1]$$

- first, multiply each input by its weight and add them up
  $w \cdot x = 0.5\times0.2 + 0.6\times0.3 + 0.1\times0.9 = 0.10 + 0.18 + 0.09 = 0.37$
- then add the bias
  $z = w \cdot x + b = 0.37 + 0.5 = 0.87$
- finally, apply the sigmoid
  $$y = \sigma(0.87) = \frac{1}{1 + e^{-0.87}} = \frac{1}{1 + 0.419} = 0.70$$

neural units

4. the results

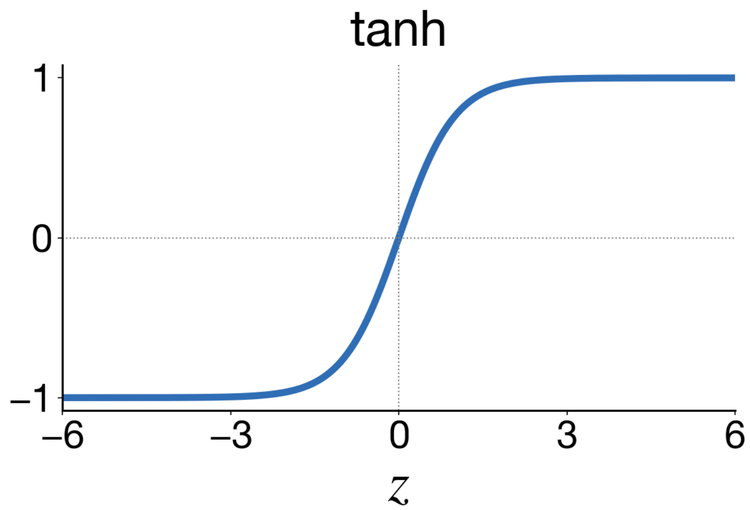
- the unit outputs 0.70
- does this look familiar?
- logistic regression is a single unit!

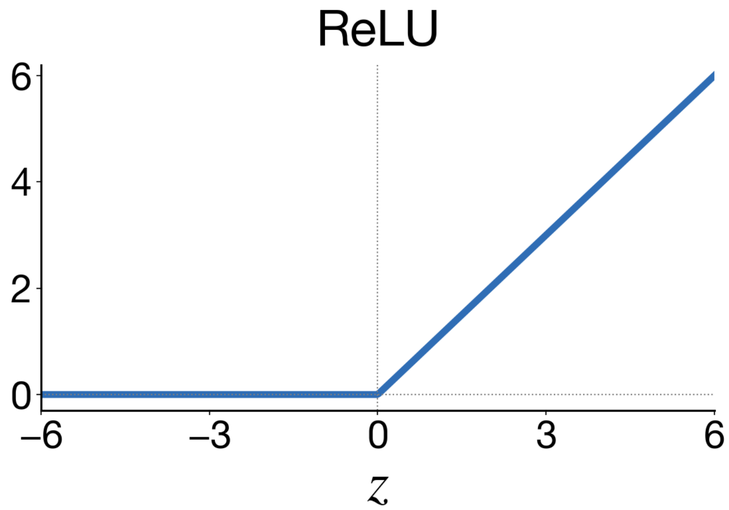


neural units

other activation functions

- the sigmoid is not the only activation function
  - **tanh** looks like the sigmoid, but goes from −1 to 1
  - **ReLU** (rectified linear unit) keeps z if it is positive, and returns 0 otherwise

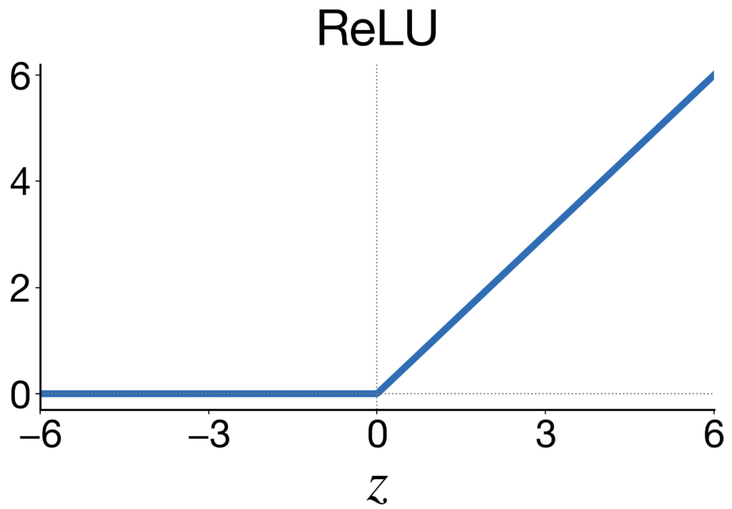




neural units

ReLU

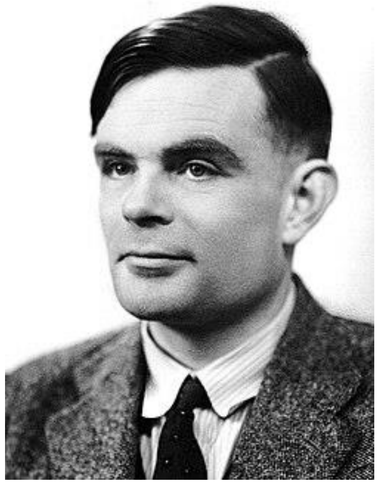
- ReLU is the most common activation function today
- why?
- sigmoid and tanh flatten out when z is large
  - where the curve is flat, the network gets almost no signal to learn from
  - this is called the **vanishing gradient** problem
- ReLU never flattens out for positive z



neural units

why more than one unit?

- a single unit is just logistic regression, which works well for many tasks
- why do we need more than one unit?
- “[M]achines of this character can behave in a very complicated manner when the number of units is large.”- Alan Turing (1948) “Intelligent Machinery”



Alan Turing

the XOR problem

the XOR problem

logical functions

- AND: output 1 when both inputs are 1
- OR: output 1 when at least one input is 1
- XOR (exclusive or): output 1 when exactly one input is 1

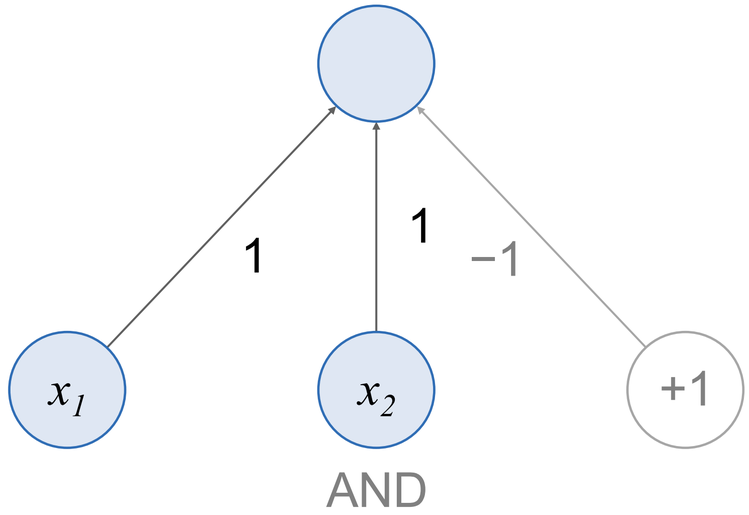
x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

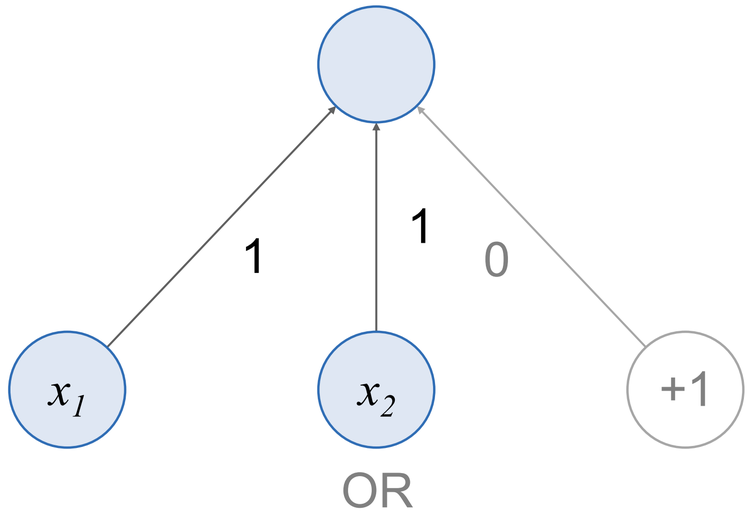
the XOR problem

the perceptron

- the **perceptron**: an simple neural unit
  - binary output (0 or 1)
  - no non-linear activation function

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$





the XOR problem

the perceptron

- let's check AND
- w = [1, 1] and b = −1

x_1	x_2	$z = x_1 + x_2 - 1$	y
0	0	−1	0
0	1	0	0
1	0	0	0
1	1	1	1

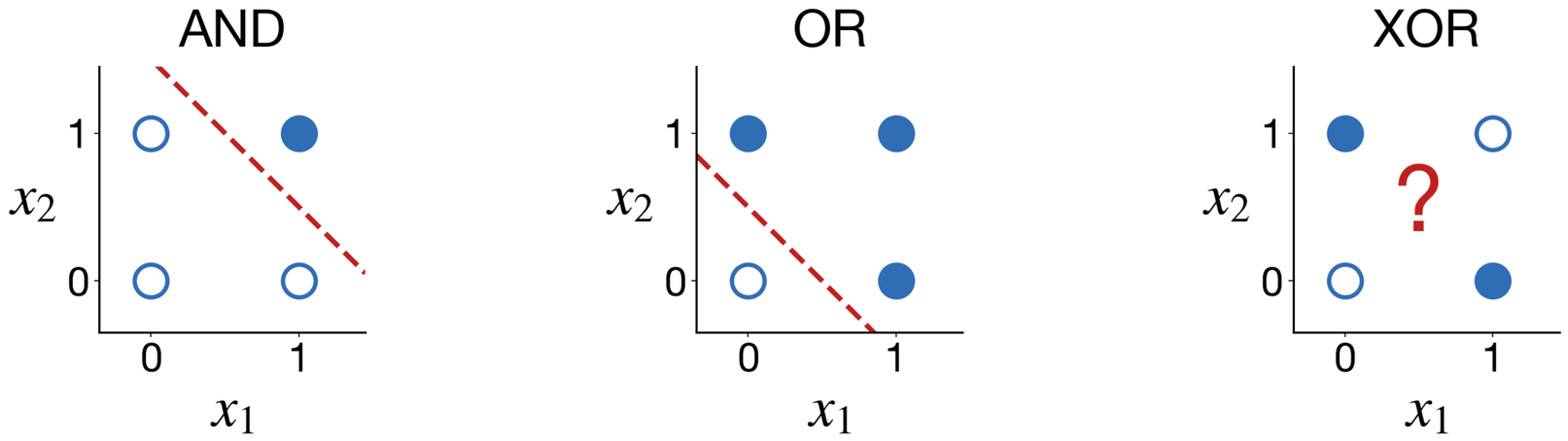
- now try: can you find w_1, w_2 and b for XOR?

x_1	x_2	z = ?	y
0	0		0
0	1		1
1	0		1
1	1		0

the XOR problem

decision boundaries

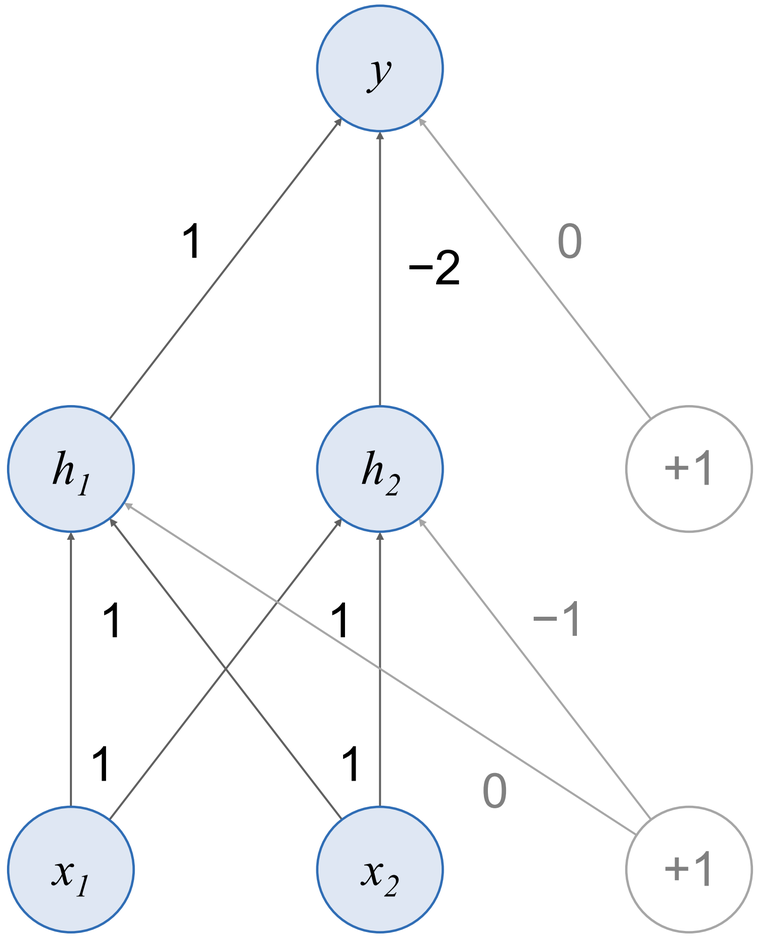
- you can't! a unit can only draw a straight line between the 0s and the 1s
  - the line is called the **decision boundary**
  - for XOR, no line has the two 1s on one side and the two 0s on the other
- we say XOR is not **linearly separable**



the XOR problem

the solution

- XOR can be computed if we stack units into layers
  - two ReLU units h_1 and h_2 form a **hidden layer**
  - one output unit y reads the hidden layer
- the numbers on the arrows are the weights

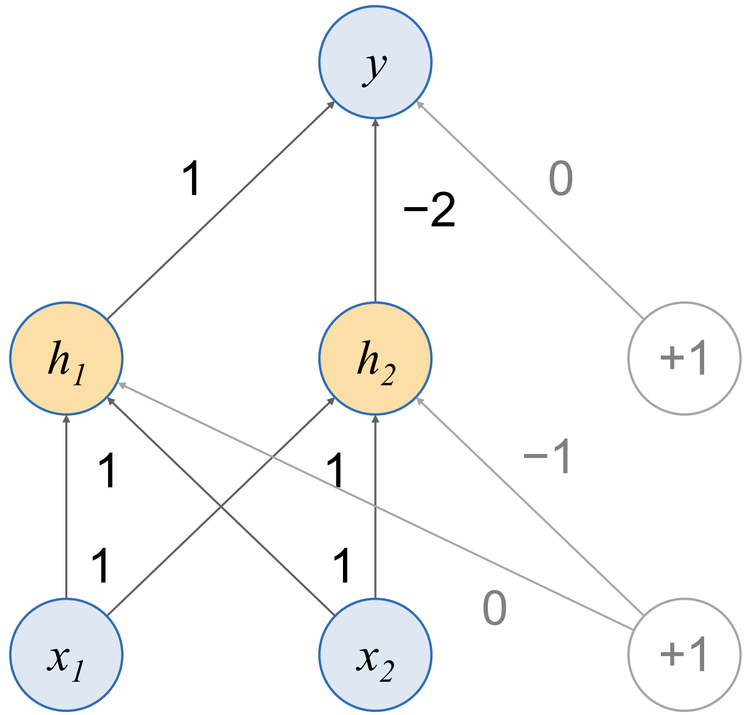


the XOR problem

1. the hidden layer

- $h_1 = x_1 + x_2 + 0$
- $h_2 = x_1 + x_2 - 1$

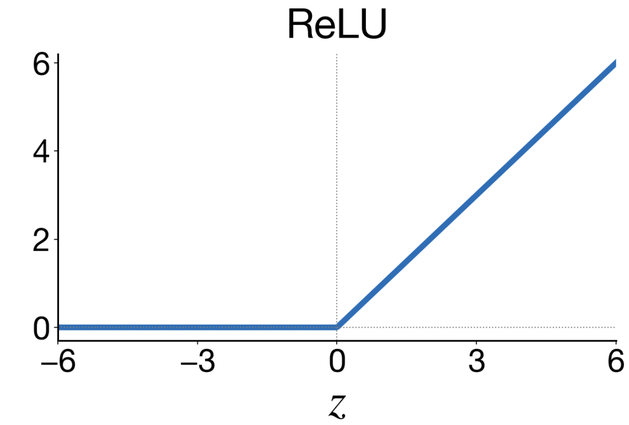
x_1	x_2	z into h_1	z into h_2	h_1	h_2	y
0	0	0	−1			
0	1	1	0			
1	0	1	0			
1	1	2	1			



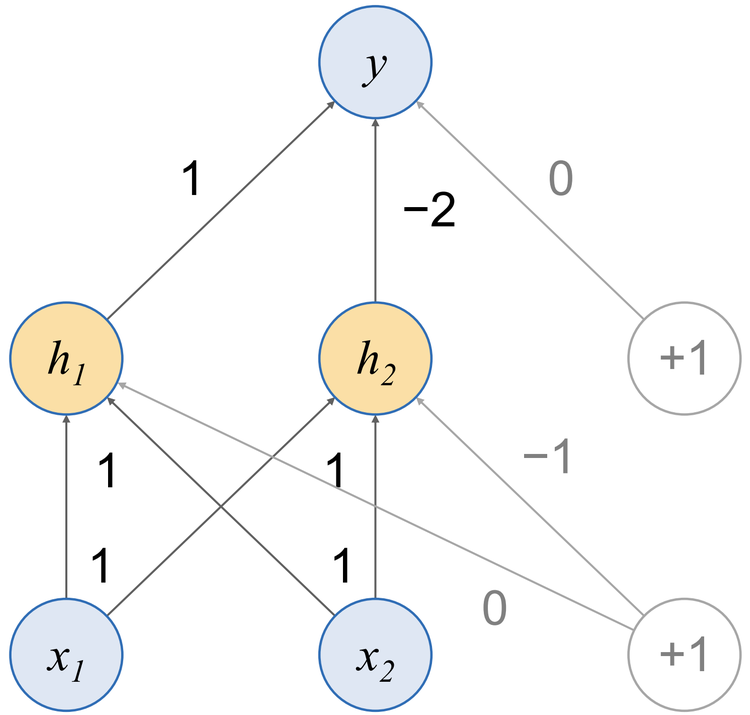
the XOR problem

2. ReLU

- then ReLU turns the negative sums into 0



x_1	x_2	z into h_1	z into h_2	h_1	h_2	y
0	0	0	−1	0	0	
0	1	1	0	1	0	
1	0	1	0	1	0	
1	1	2	1	2	1	

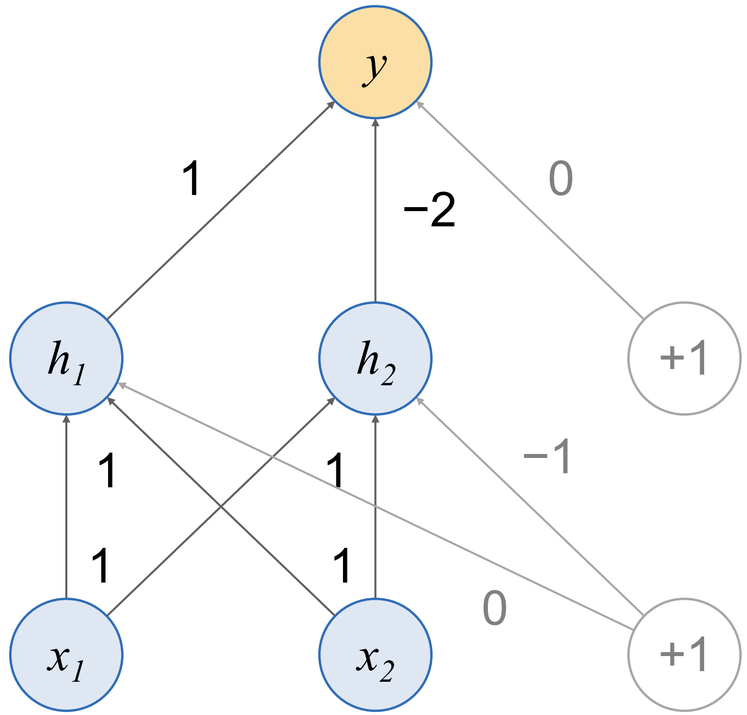


the XOR problem

3. the output unit

- $y = h_1 - 2h_2$
- that's XOR!

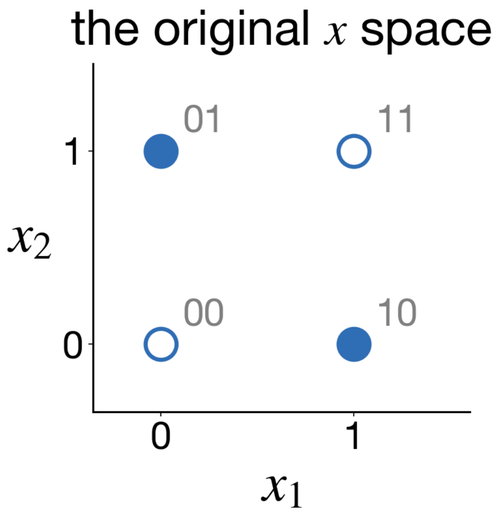
x_1	x_2	z into h_1	z into h_2	h_1	h_2	y
0	0	0	−1	0	0	0
0	1	1	0	1	0	1
1	0	1	0	1	0	1
1	1	2	1	2	1	0

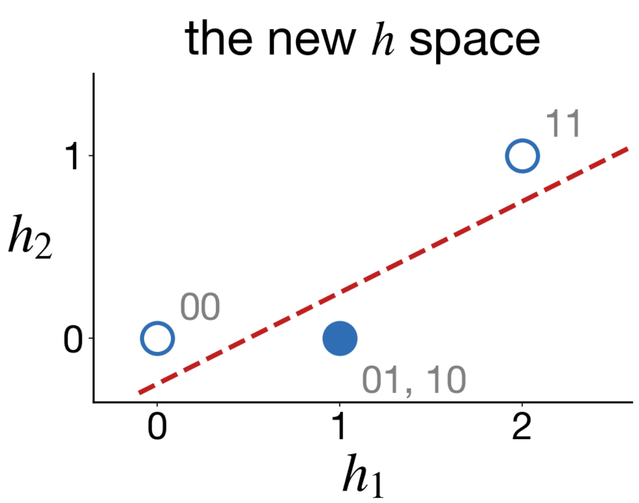


the XOR problem

the hidden representation

- why did this work?
  - 01 and 10 both became h = [1, 0]
  - in h space, a single line separates them from 00 and 11





the XOR problem

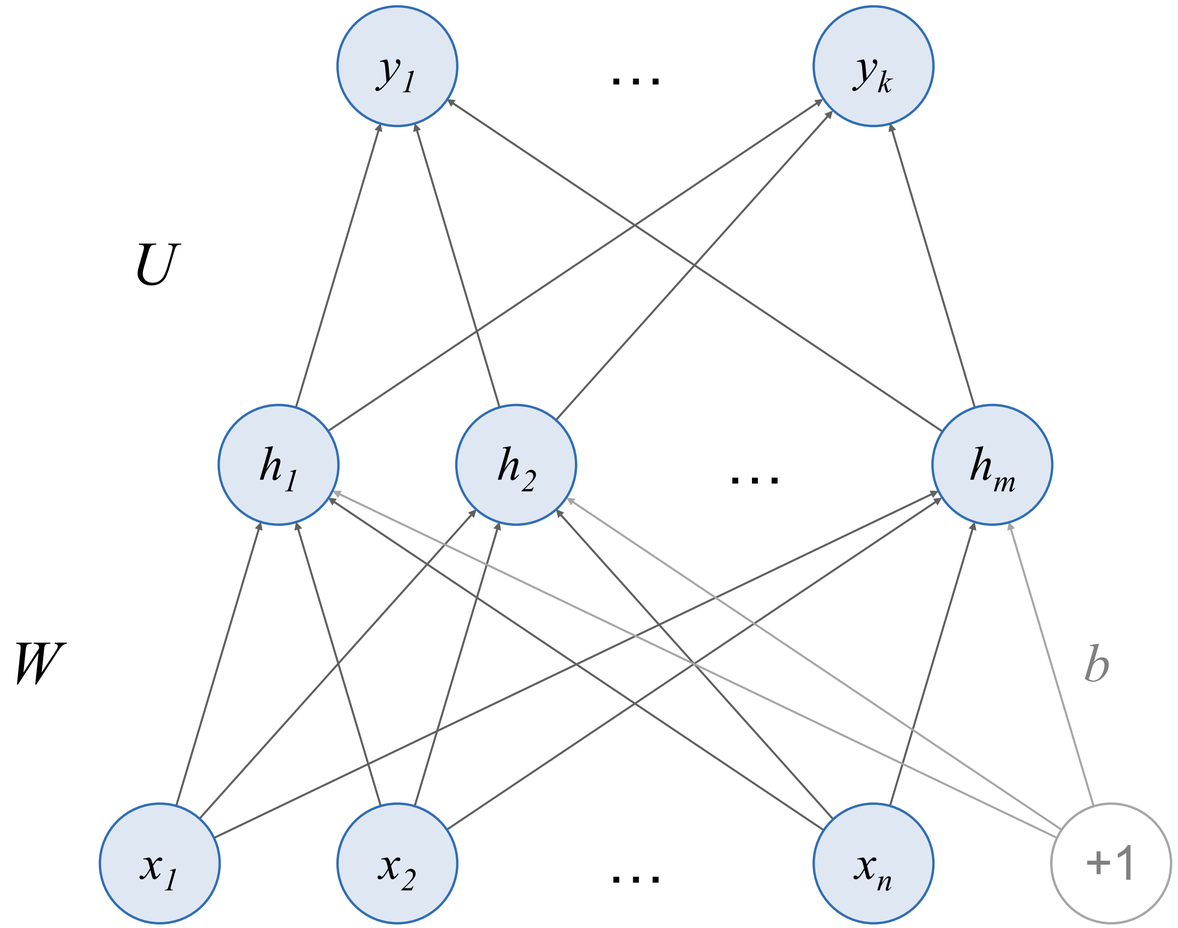
representation learning

- the hidden layer re-represented the input, which made the problem easy
- here we picked the weights by hand, but in a real network they are learned from data

feedforward networks

feedforward networks

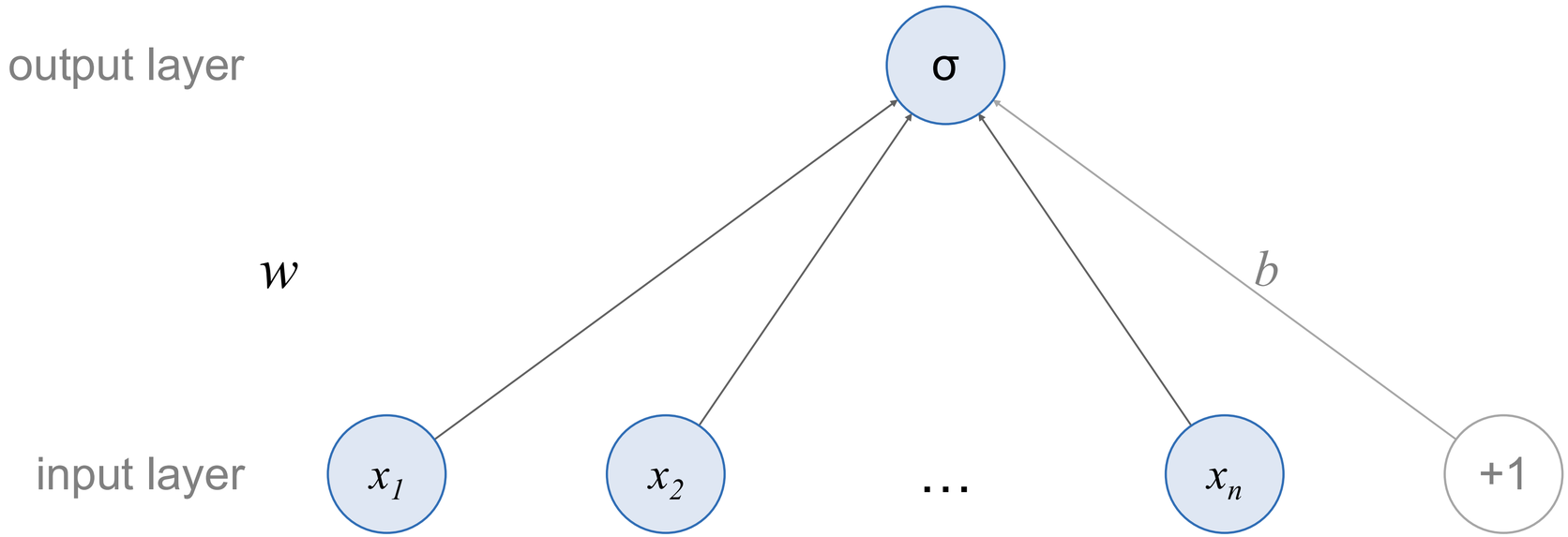
- the XOR network is a small **feedforward network**
  - the units are arranged in layers
  - each layer feeds the next one, and nothing flows backward
- three kinds of units: **input** x, **hidden** h and **output** y
- each unit is connected to every unit in the layer before it



feedforward networks

a 1-layer network

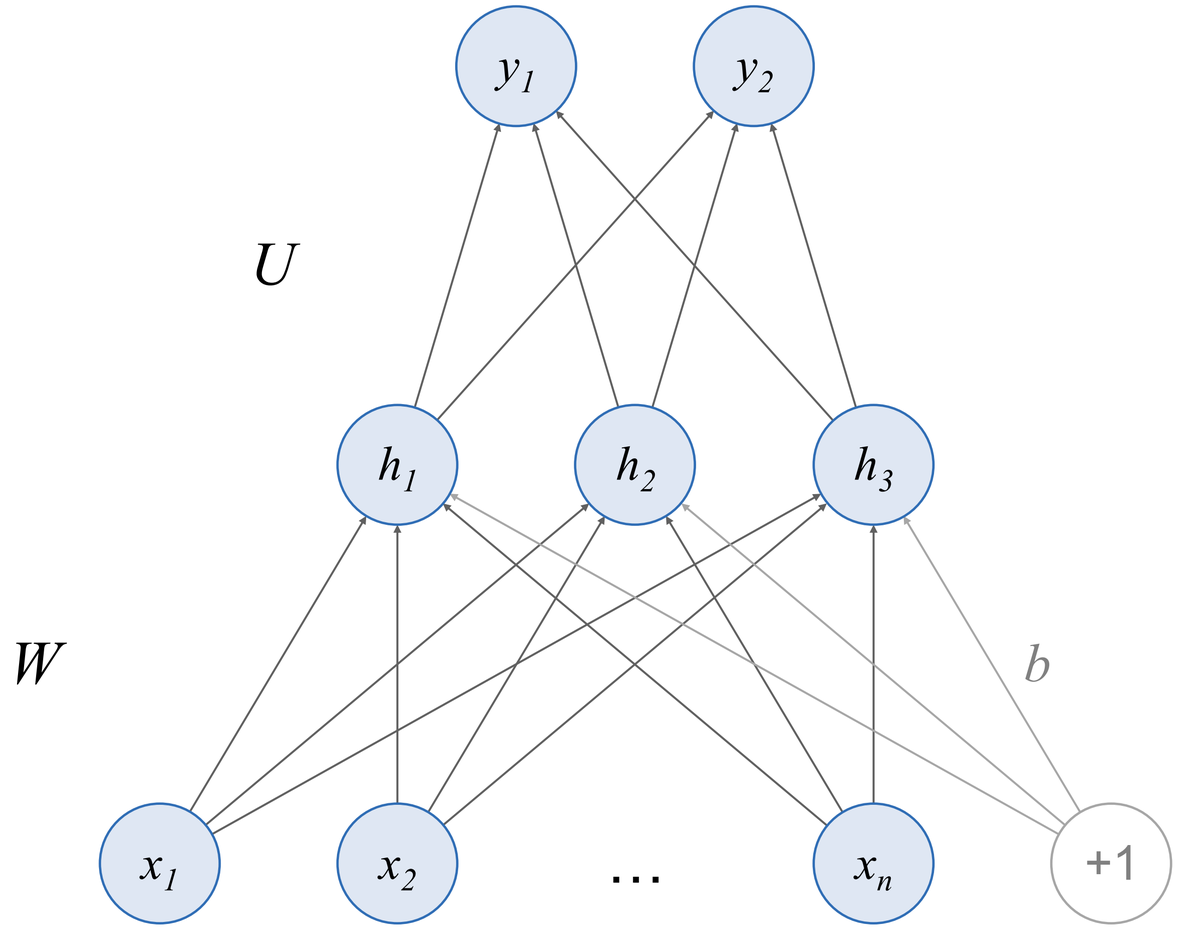
- let's build one up, starting from what we know
  - logistic regression is a network with one layer
  - we don't count the input layer



feedforward networks

a 2-layer network

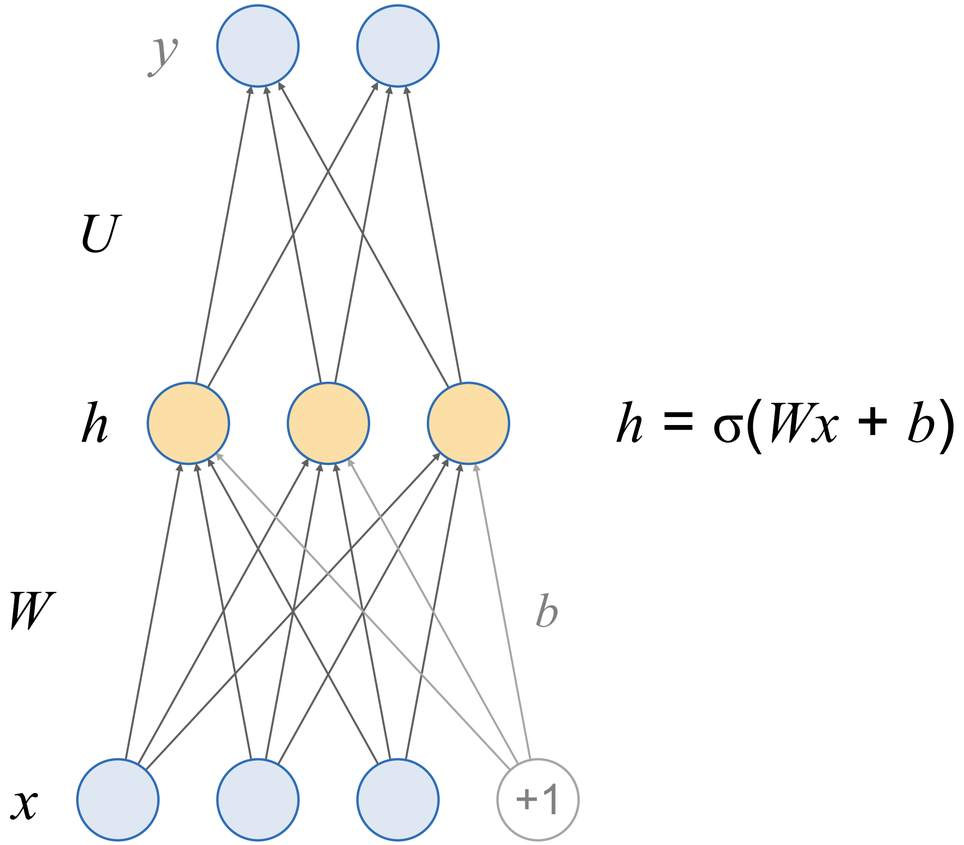
- now add a hidden layer in the middle
  - each hidden unit has its own weights and its own bias
  - we stack them into a **weight matrix** W and a bias vector b
- the output layer has its own weight matrix, U



feedforward networks

the hidden layer

- the hidden layer does what a single unit does, for all of its units at once
  $h = \sigma(Wx + b)$
- in the XOR network, the hidden layer had these weights and biases
  $W = [[1,1],[1,1]], b = [0,-1]$

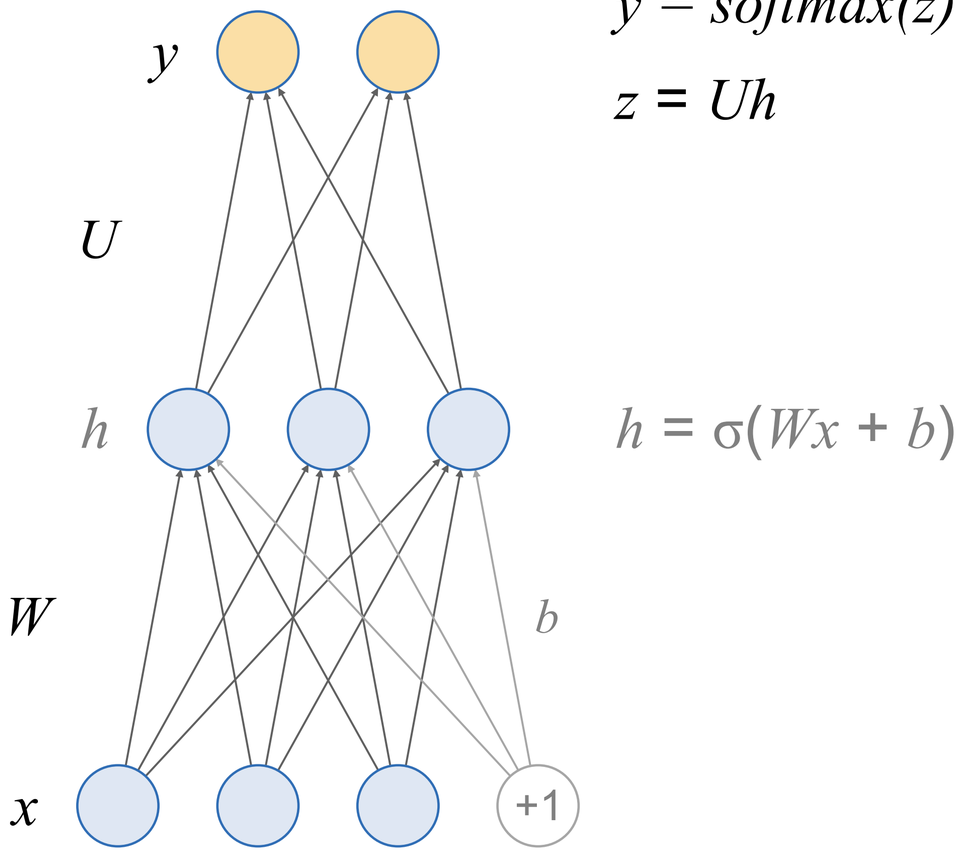


feedforward networks

the output layer

- the output layer takes h as its input, and computes one score for each class
  $z = Uh$
- but for classification we want probabilities, not scores
  - the **softmax** turns the scores into probabilities that add up to 1

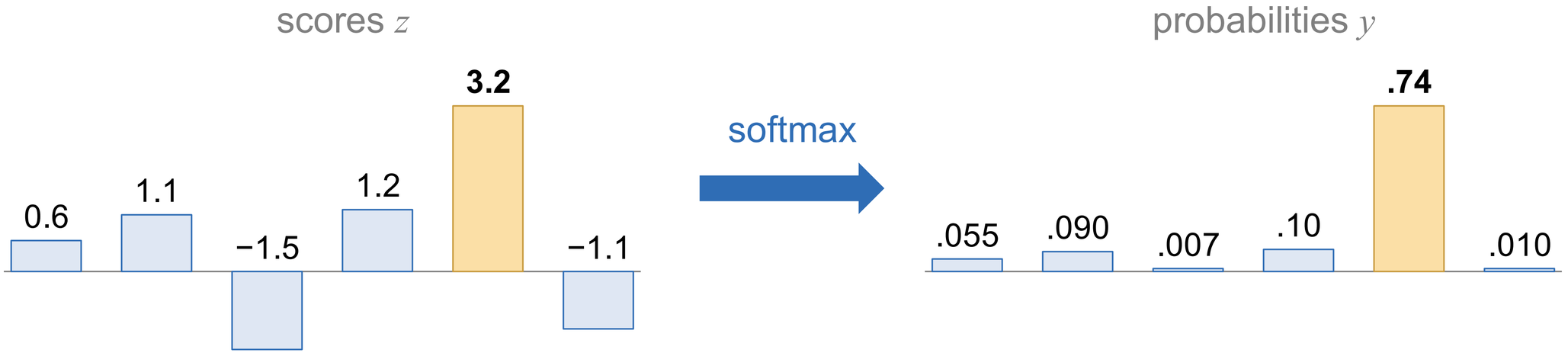
$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$



feedforward networks

the softmax

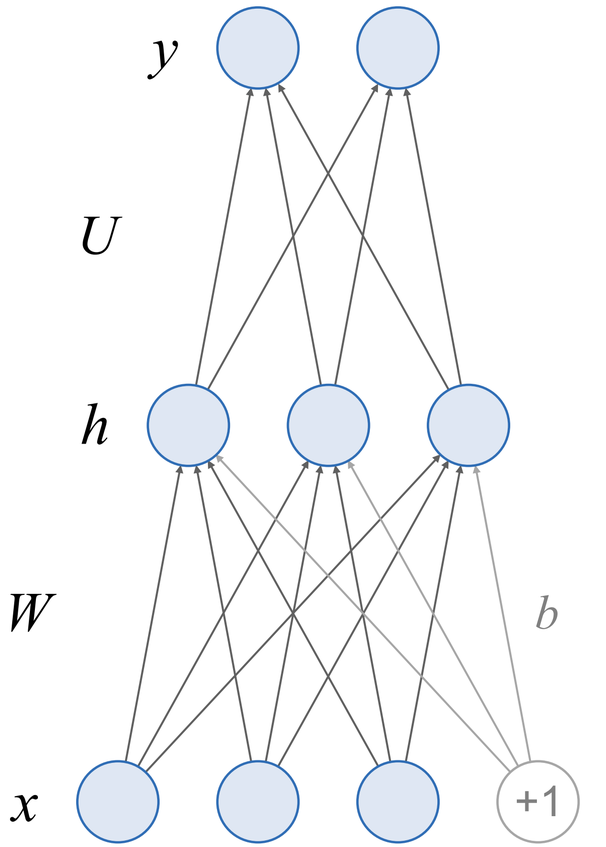
- the softmax is like the sigmoid, but for more than two classes
  $z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$
  $\text{softmax}(z) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$
- the highest score gets most of the probability, but not all of it
- the scores that go into the softmax are called **logits**



feedforward networks

the whole network

- why does the hidden layer need the non-linear σ?
- without it, the two layers collapse into one
  - $U(W\,x + b)$ is just another $W'x + b'$



y = softmax(z)

$z = Uh$

$h = \sigma(Wx + b)$

feedforward networks

deeper networks

- a deeper network repeats the same two steps for every layer
  $z^{[1]} = W^{[1]}x + b^{[1]}$
  $a^{[1]} = g^{[1]}\left(z^{[1]}\right)$
  $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$
  $a^{[2]} = g^{[2]}\left(z^{[2]}\right) = \hat{y}$
- the number in brackets is the layer, and g is its activation function

feedforward networks

what to do with them?

- we now have a basic neural network
  - there are other kinds, but they are built from the same pieces
- let's try it on the two tasks we already know from the class
  - text classification (week 8)
  - language modeling (week 7)
- current best systems use bigger networks, but these two simple models introduce the important ideas

classification

classification

sentiment analysis

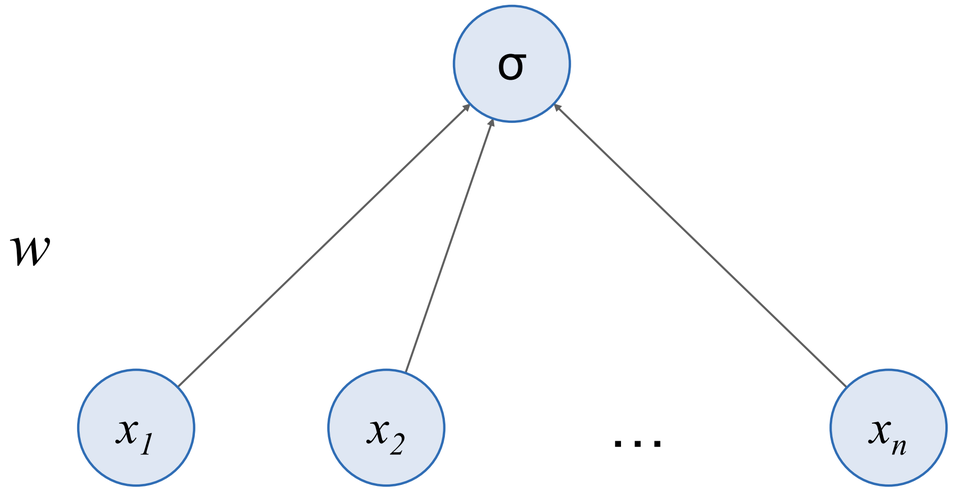
- the first task: is this review positive or negative?
  - the input layer is the same six hand-built features

var	definition
x_1	count of positive lexicon words in the review
x_2	count of negative lexicon words in the review
x_3	1 if *no* is in the review, 0 otherwise
x_4	count of 1st and 2nd person pronouns
x_5	1 if “!” is in the review, 0 otherwise
x_6	log(word count of the review)

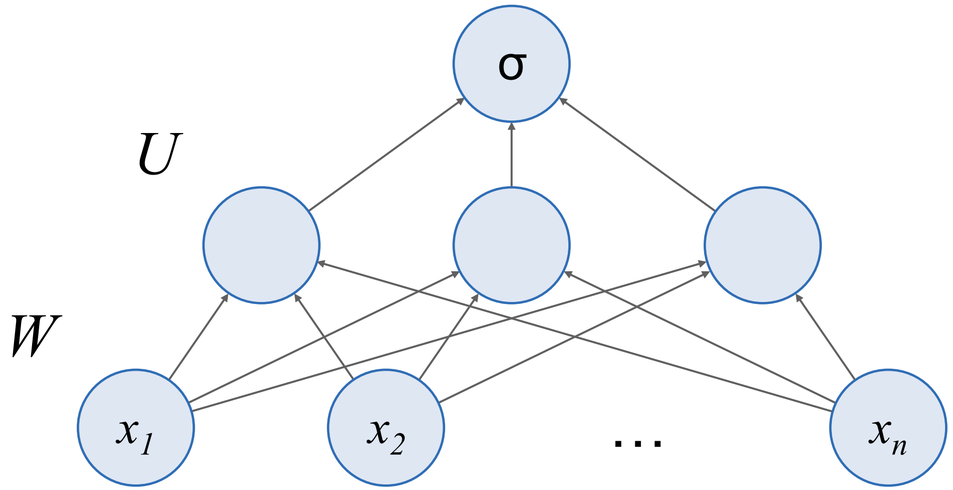
classification

adding a hidden layer

- what does a hidden layer add?
  - logistic regression weighs each feature separately
  - a hidden layer lets the features interact, like *no* next to a positive word



logistic regression

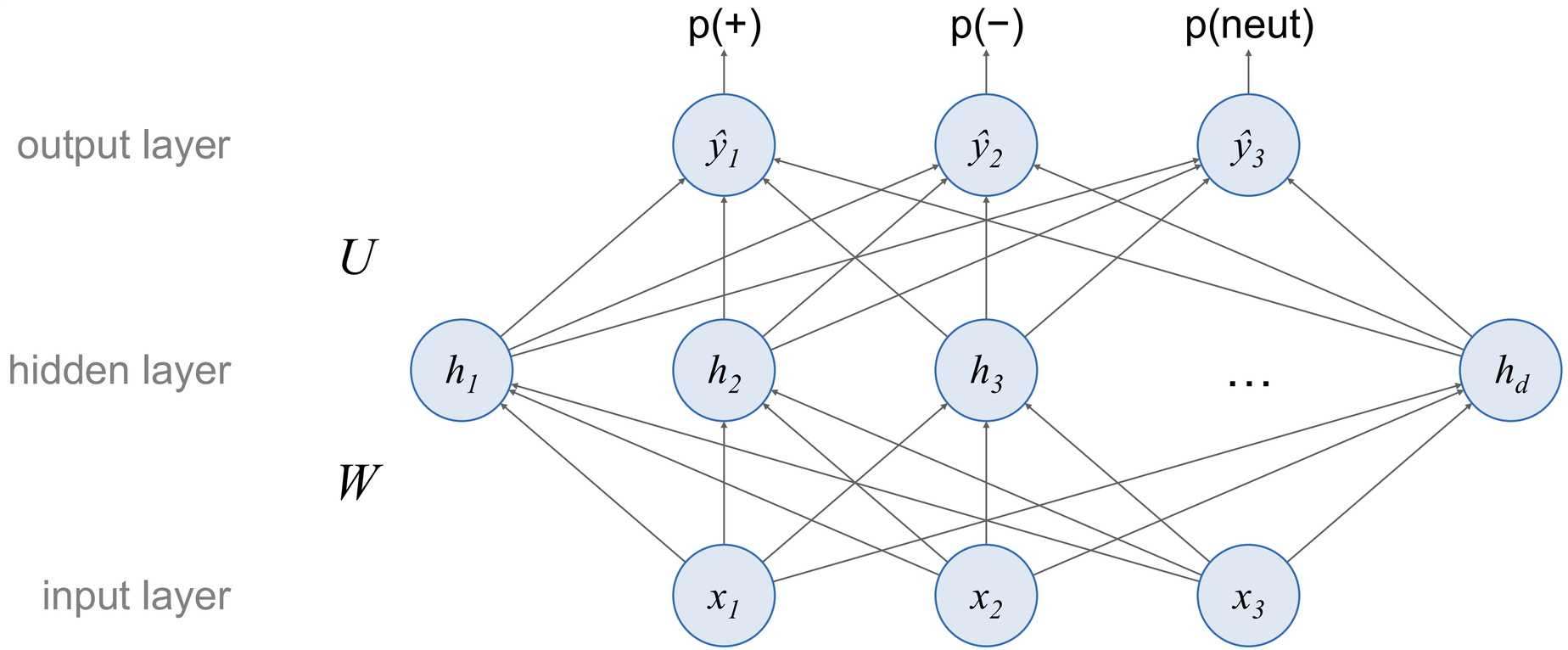


a 2-layer network

classification

the network

- three features go in, and three probabilities come out: positive, negative and neutral

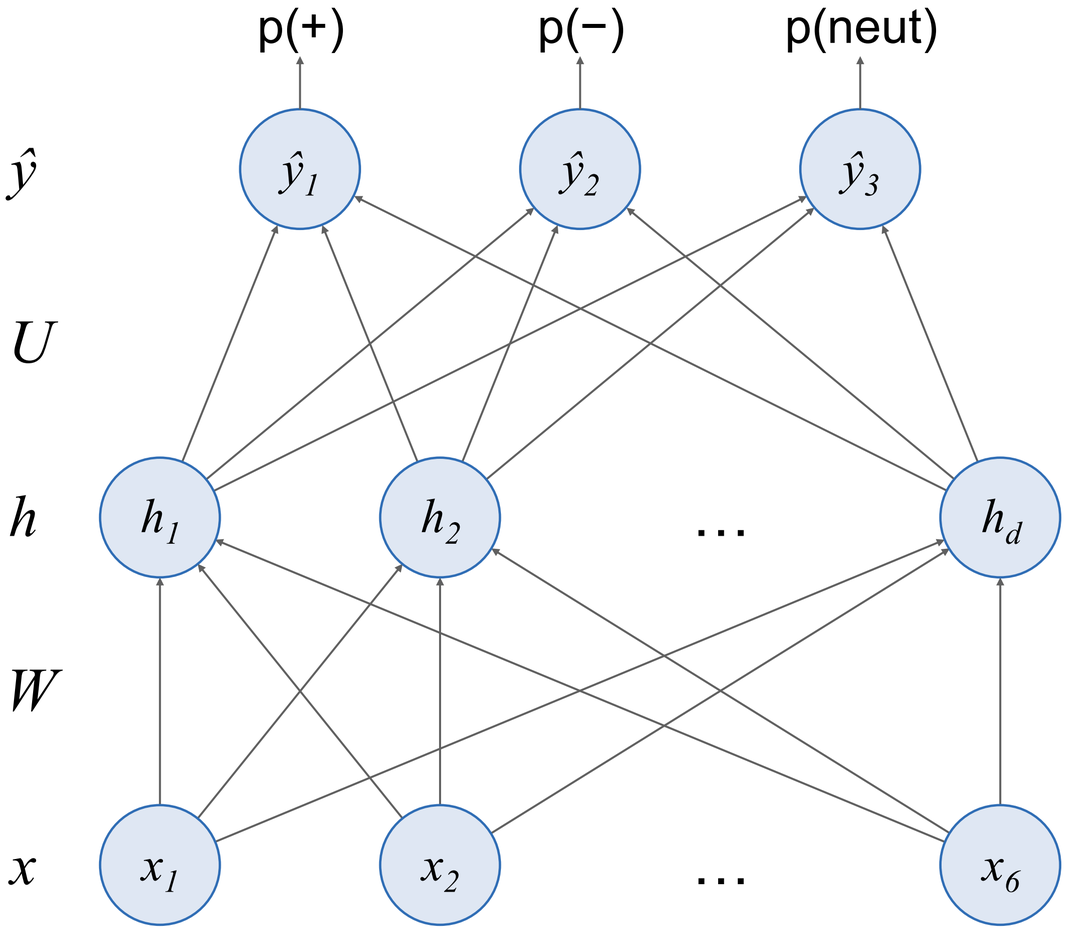


classification

the equations

$$x = [x_1, x_2, \ldots, x_6]$$
$$h = \sigma(Wx + b)$$
$$z = Uh$$
$$\hat{y} = \text{softmax}(z)$$

- at the output, $\hat{y}$ has one probability for each class



classification

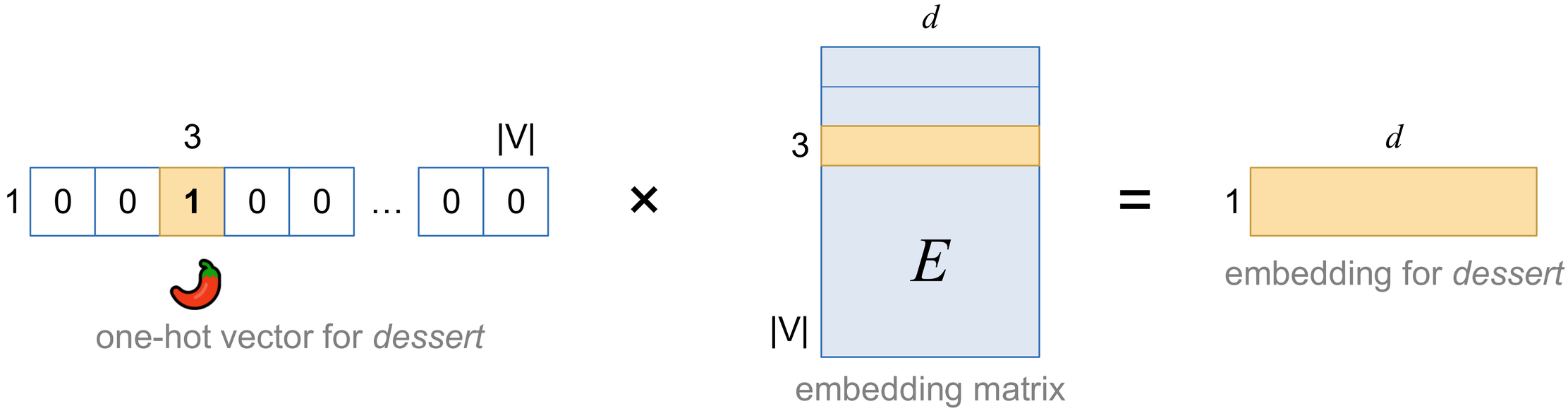
embeddings as input

- but a network can learn its own features, so let's drop the hand-built ones
  - the input is the words themselves, as the embeddings from last week
- the **embedding matrix** E has one row for each word in the vocabulary
- for *dessert was great*, we look up each word's row in E
  - the words have indices 3, 9824 and 226, so we take rows 3, 9824 and 226

classification

embeddings as input

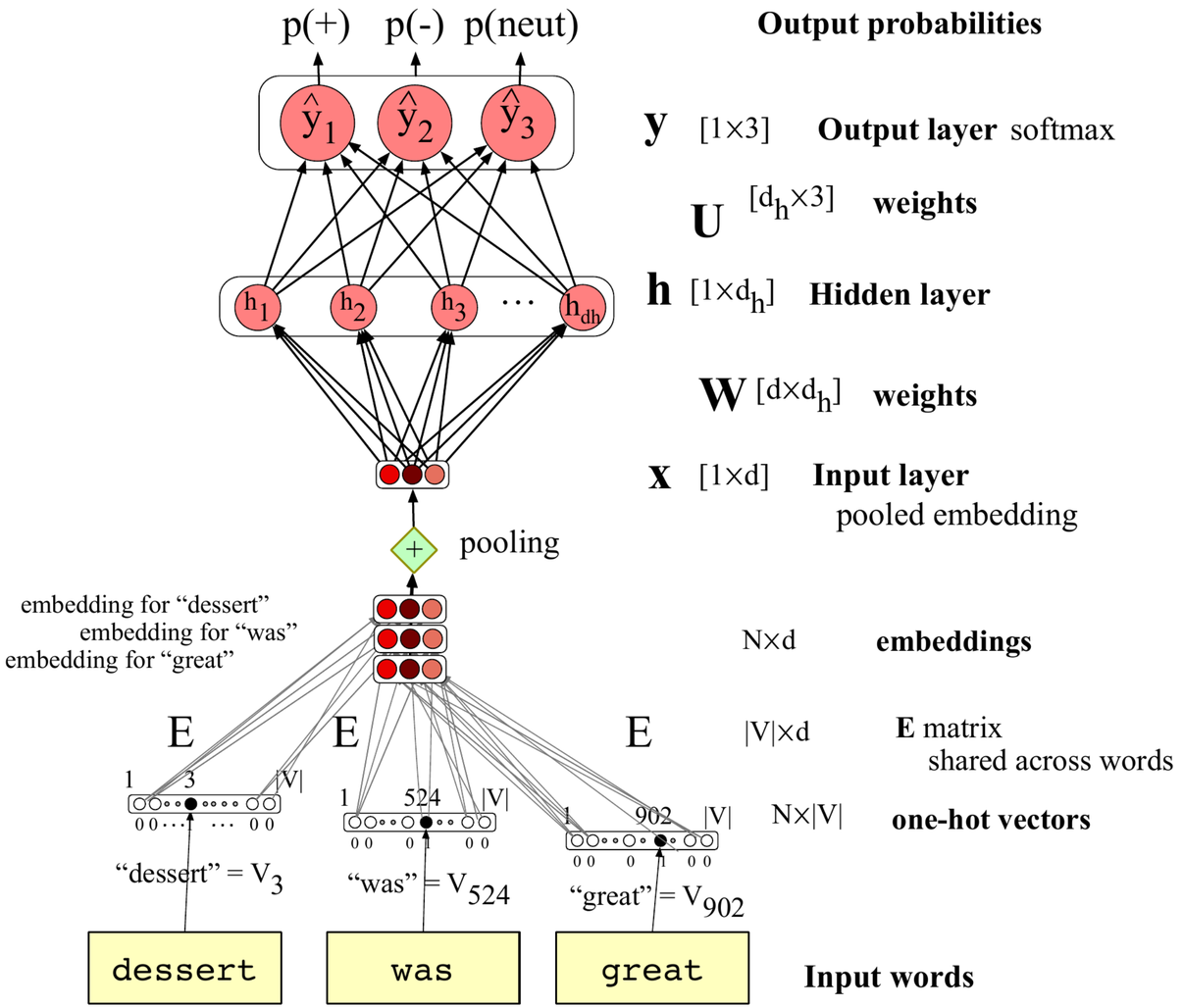
- another way to see the lookup: write each word as a **one-hot vector**
  - all zeros, except a 1 at the word's index
  - multiplying it by E selects the embedding for *dessert*



classification

pooling

- now we have three embeddings, but the network takes one input vector
  - and reviews come in different lengths
- so we **pool**: we average the embeddings into a single vector $x = \text{mean}\big(e(w_1), e(w_2), \ldots, e(w_N)\big)$



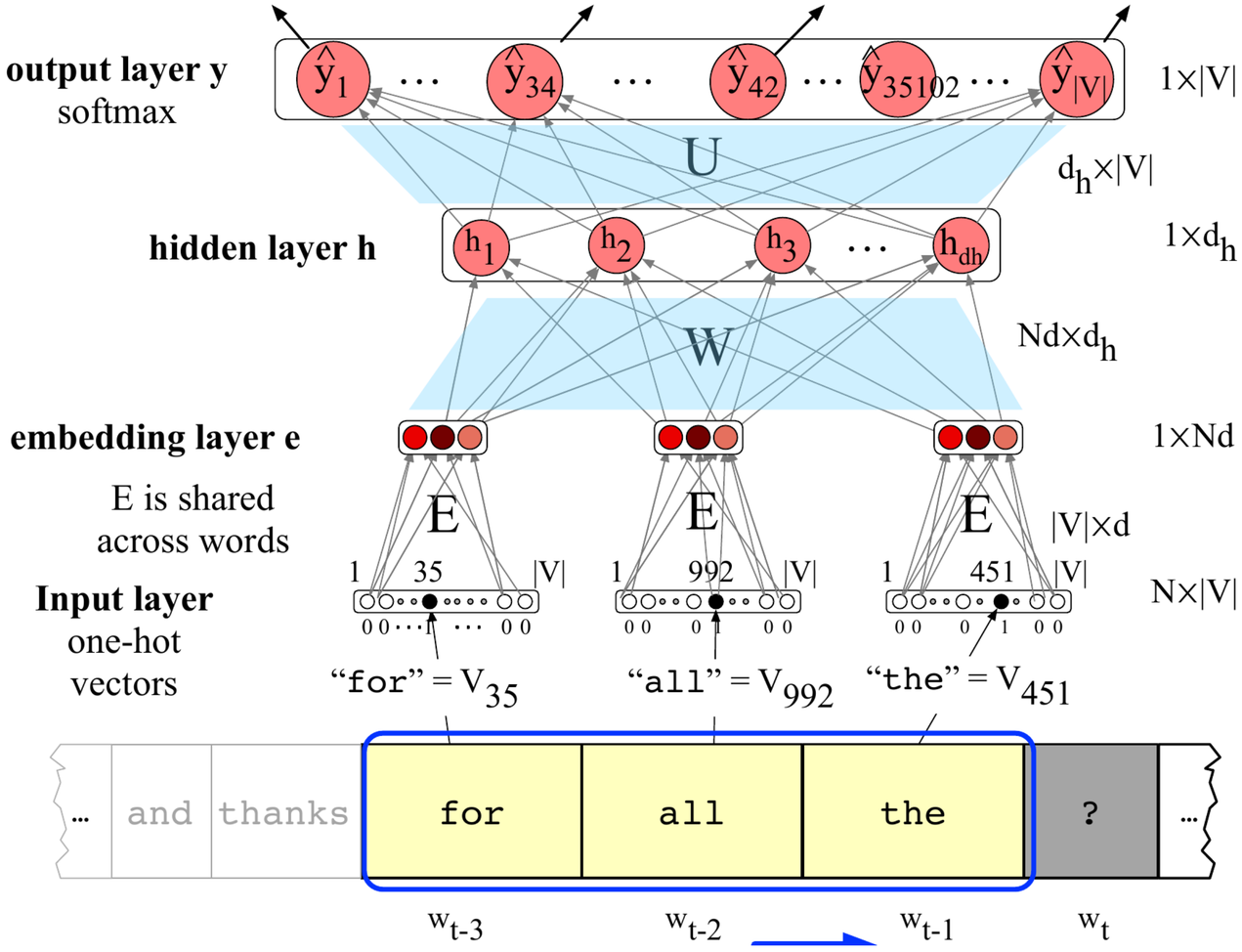
neural language models

neural language models

- the second task: language modeling
  - *and thanks for all the* ___?
  - we want the probability of the next word, given the words before it
- in week 7 we used n-gram counts
- now we can try a feedforward network
- like an n-gram model, we have the conditional independence assumption $P(w_t|w_1, \ldots, w_{t-1}) \approx P(w_t|w_{t-N+1}, \ldots, w_{t-1})$

neural language models

- the last three words go in as embeddings
  - word order matters here, so we **concatenate** the embeddings instead of pooling them
- a probability for every word in the vocabulary comes out



neural language models

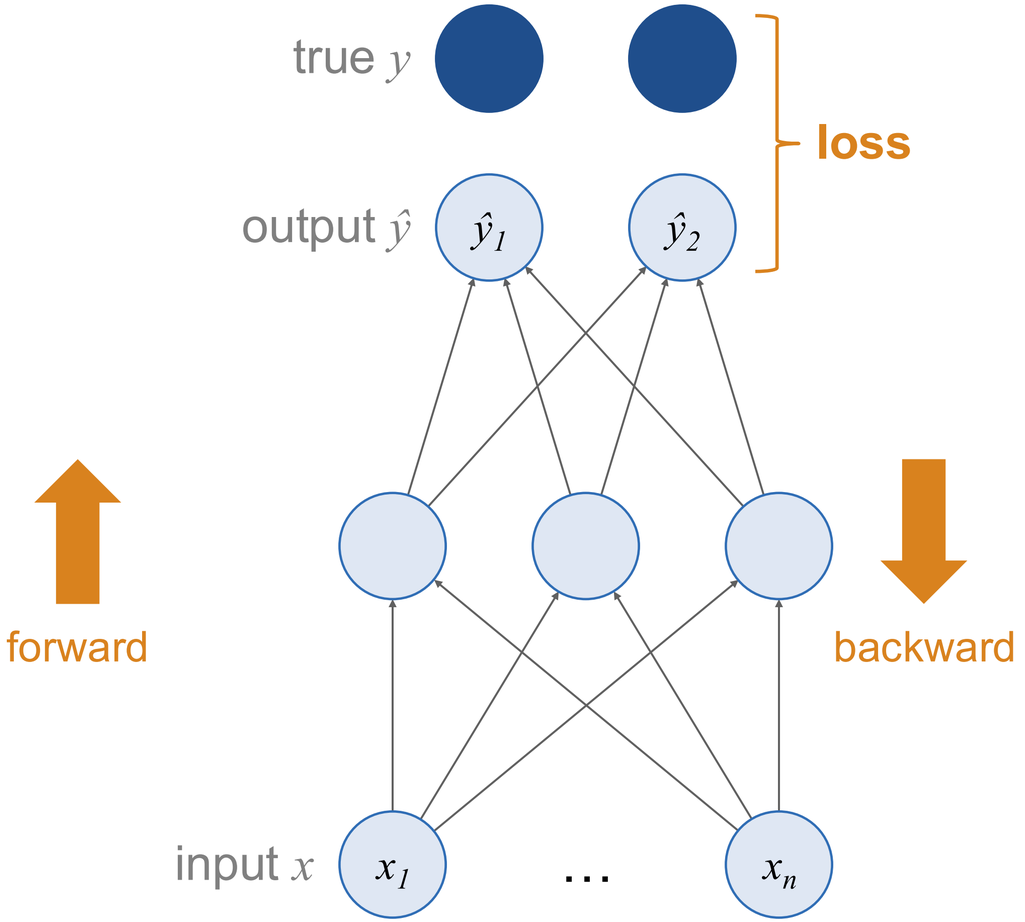
comparison with n-grams

- suppose the training data has *I have to make sure that the cat gets fed*
  - but never *dog gets fed*
- at test time: *I forgot to make sure that the dog gets* ___
- an n-gram model can't predict *fed*, because it has never seen *dog gets fed*
- a neural model can: *cat* and *dog* have similar embeddings, so what it learned about *cat* carries over to *dog*

neural language models

training

- one thing is still missing: where do the weights come from?
- they are learned!
- the **loss** measures how wrong the output is
  - gradient descent nudges each weight to make the loss smaller
- **backpropagation** carries the error from the output back to every layer



neural language models

what's still missing?

- our language model has a short memory: it only sees the last three words
- *The chicken didn't cross the road because it was too* ___
  - to guess the next word, we need to know what *it* is
  - but *chicken* and *road* are outside the window
- and a bigger window needs a separate set of weights for every position

neural language models

from here to large language models

- **attention** lets each word look at all the words before it (Thursday)
  - a network built from attention is a **transformer**
- concepts reused from today: units, layers, embeddings, the softmax, training

	today's language model	**GPT-3 (2020)**
can look at	the last 3 words	the last 2,048 tokens
layers	2	96
parameters	millions	175 billion

next time

- Thursday, Nov 12
  - large language models
- **HW 6 due Thursday, Nov 12, 11:59 pm**